Отфильтрованные записи

Что требуется: показать страницу, на которой отображаются только записи, подходящие под условие.

Открой на компьютере: `/productions/`. Это список постановок.

В фильтре «Жанр» выбери **«комедия»** и нажми «Показать». В исходных данных останется **«Ревизор»**.

Выбери **«драма»** и снова нажми кнопку. Теперь будет **«Вишнёвый сад»**. При сбросе фильтра видны обе постановки.

Условие отбора находится в `sql/queries.sql`, запрос `productions_by_genre`:

```
SELECT * FROM productions
WHERE genre = ?
ORDER BY premiere_date, id;
```

`WHERE` оставляет только подходящие строки. Вместо `?` передаётся выбранный жанр.

Как фильтр доходит до страницы

1. После выбора жанра браузер передаёт параметр `genre`. Например:

```
/productions/?genre=комедия
```

2. В `repertoire/views.py`, строка 13, представление читает этот параметр:

```
genre = request.GET.get("genre", "").strip()
```

3. В строке 18 вызывается запрос. Главное обращение, с переносами для чтения:

```
fetch(
    conn,
    "productions_by_genre",
    (genre,),
)
```

4. Найденные строки преобразуются в объекты постановок. Список передаётся в шаблон под именем `productions`.

5. Шаблон перебирает этот список и показывает карточки. В браузер попадают только отобранные постановки.

Связанные данные на одной странице

Что требуется: показать страницу со сведениями из нескольких связанных таблиц и объяснить использование JOIN.

Открой на компьютере: `/shows/`. Здесь рядом показаны название спектакля, дата, зал и свободные места.

Откуда берутся поля

`productions` хранит постановки, в том числе название `title`.

`shows` хранит показы: дату `datetime`, зал `hall` и ссылку `production_id` на постановку.

Одна постановка может иметь несколько показов. Поле `shows.production_id` указывает, к какой постановке относится конкретный показ.

В исходных данных: «Вишнёвый сад» - 20.10.2026, «Ревизор» - 25.11.2026. Оба показа проходят в Большом зале.

JOIN соединяет сведения о показе с названием его постановки.

Как работает соединение JOIN

Открой `sql/queries.sql`, строку 23: запрос `schedule`. Он выполняется для страницы расписания.

Упрощённый вариант того же соединения:

```
SELECT s.datetime, s.hall, p.title
FROM shows AS s
JOIN productions AS p
    ON p.id = s.production_id;
```

`s` - короткое имя таблицы показов. `p` - короткое имя таблицы постановок.

`ON p.id = s.production_id` означает: к показу присоединяется именно та постановка, чей ID указан в этом показе.

В результате одна строка содержит дату и зал из `shows` и название из `productions`.

Полный запрос в проекте дополнительно присоединяет активные брони через `LEFT JOIN bookings` и считает свободные места. Показ без броней тоже остаётся в расписании.

Как JOIN попадает в шаблон

1. В `performances/views.py`, функция `schedule_rows`, выполняется:

```
fetch(conn, "schedule")
```

2. Полученные строки сохраняют имена полей. Дата преобразуется в объект Python для форматирования.

3. Представление `shows` передаёт список в контекст:

```
"shows": schedule_rows(data, conn)
```

4. Шаблон `templates/includes/schedule.html` выводит поля каждой строки:

```
{{ show.title }}
{{ show.datetime|date:"d.m.Y H:i" }}
{{ show.hall }}
```

Что сказать: «JOIN связывает показы с постановками по production_id. Название берётся из постановки, дата и зал - из показа. Результат запроса передаётся в шаблон и выводится в таблице».